

Тема: Python, Алгоритмы и Структуры данных

Функции, Рекурсия, Строки, Файлы, Сортировки и Графы

1. ФУНКЦИИ И РЕКУРСИЯ В PYTHON

Функции (def)

Функция — фрагмент кода, выполняющий задачу при вызове.

```
def power(a, b=2): # b - дефолтный аргумент
    return a ** b
```

```
res = power(5) # 25
```

- `return` завершает работу функции и возвращает значение.
- **Локальные переменные** видны только внутри функции.

Рекурсия

Вызов функции из самой себя. Требуется **базовое условие** выхода!

```
def fact(n):
    if n <= 1: return 1 # Базовый случай
    return n * fact(n - 1) # Рекурсивный шаг
```

- Переполнение стека: `RecursionError`.

2. РАБОТА СО СТРОКАМИ (STRINGS)

Строки в Python — **неизменяемый (immutable)** упорядоченный тип данных.

Метод / Операция	Пример кода	Результат / Назначение
Срезы (Slicing)	<code>s = "ENT2026"[::-1]</code>	"6202TNE" (реверс строки). Формат: <code>[start:stop:step]</code>
<code>split(sep)</code>	<code>"a,b,c".split(",")</code>	Разбивает строку в список <code>['a', 'b', 'c']</code> по разделителю
<code>join(iterable)</code>	<code>"-".join(['A', 'B'])</code>	Объединяет элементы в строку: "A-B"
<code>find()</code> / <code>index()</code>	<code>"python".find("th")</code>	Возвращает индекс первого вхождения (2) или -1 если не найдено
<code>replace(old, new)</code>	<code>"123".replace("2", "5")</code>	Заменяет подстроку: "153"

3. РАБОТА С ФАЙЛАМИ (FILE I/O)

Режимы открытия файлов

- `'r'` — только чтение (ошибка, если файла нет).
- `'w'` — запись (создает новый или ****перезаписывает****).
- `'a'` — добавление в конец файла (append).

Конструкция with open

```
# Безопасное чтение файла по строкам
with open("data.txt", "r", encoding="utf-8") as f:
    lines = f.readlines() # Список всех строк
    # f.read() - весь файл одной строкой
    # f.readline() - одна строка
```

Важно для ЕНТ: Оператор `with` автоматически закрывает файл после завершения блока кода, исключая утечку ресурсов и потерю данных!

4. АЛГОРИТМЫ СОРТИРОВКИ (SORTING ALGORITHMS)

Сортировка — процесс упорядочивания элементов массива по возрастанию или убыванию.

Алгоритм	Сложность (Worst / Avg)	Принцип работы
Пузырьковая (Bubble Sort)	$O(n^2)$ / $O(n^2)$	Попарное сравнение соседних элементов и их обмен. Наибольший «всплывает» в конец.
Выбором (Selection Sort)	$O(n^2)$ / $O(n^2)$	Поиск минимального элемента в неотсортированной части и обмен с первым неотсортированным.
Вставками (Insertion Sort)	$O(n^2)$ / $O(n^2)$	Элемент извлекается и вставляется на правильную позицию в уже отсортированную часть.
Быстрая (Quick Sort)	$O(n^2)$ / $O(n \log n)$	Разделяй и властвуй: выбор опорного элемента (pivot) и разбиение массива на две части.
Слиянием (Merge Sort)	$O(n \log n)$ / $O(n \log n)$	Рекурсивное деление массива пополам и последующее слияние отсортированных частей.

Сортировка пузырьком (Python)

```
def bubble_sort(arr):
    n = len(arr)
    for i in range(n - 1):
        for j in range(n - i - 1):
            if arr[j] > arr[j + 1]:
                arr[j], arr[j + 1] =
arr[j + 1], arr[j]
    return arr
```

Встроенные методы Python

```
# Сортировка на месте (меняет исходный)
lst = [3, 1, 4]
lst.sort(reverse=True) # [4, 3, 1]

# Создание нового отсортированного списка
new_lst = sorted("python") #
['h', 'n', 'p', 'o', 't', 'y']
```

5. ТЕОРИЯ ГРАФОВ (GRAPH THEORY)

Граф $G = (V, E)$ состоит из множества вершин (V , Vertices) и рёбер (E , Edges).

Основные понятия графов

- **Ориентированный (Орграф):** Рёбра имеют направление (стрелки).
- **Взвешенный:** Каждому ребру присвоен вес (стоимость/расстояние).
- **Степень вершины:** Количество рёбер, инцидентных ей.
- **Путь:** Последовательность вершин, соединенных рёбрами.

Способы представления в Python

```
# 1. Матрица смежности (Adjacency Matrix)
# A[i][j] = 1, если есть ребро между i и j
matrix = [
    [0, 1, 0],
    [1, 0, 1],
    [0, 1, 0]
]

# 2. Список смежности (Adjacency List)
graph = {
    0: [1],
    1: [0, 2],
    2: [1]
}
```

Алгоритмы обхода графов

Алгоритм	Структура данных	Принцип работы и применения
DFS (Поиск в глубину)	Стек (Stack) / Рекурсия	Идет вглубь до упора, затем возвращается (backtracking). Изучение компонентов связности.
BFS (Поиск в ширину)	Очередь (Queue, FIFO)	Обходит граф по слоям (волнами). Поиск кратчайшего пути в невзвешенном графе!

6. ПРАКТИЧЕСКИЕ ЗАДАНИЯ ЕНТ (10 ВОПРОСОВ)

1. Что выведет код: `print("ЕНТ_2026"[1:6:2])` ?

A) HT2

B) H_0

C) ET0

D) H_2

2. Какая временная сложность у быстрой сортировки (Quick Sort) в среднем случае?

A) $O(n)$

B) $O(n^2)$

C) $O(n \log n)$

D) $O(\log n)$

3. Какой режим открытия файла удаляет всё его текущее содержимое и записывает новые данные?

A) 'r'

B) 'a'

C) 'x'

D) 'w'

4. Какая структура данных используется при реализации поиска в ширину (BFS) в графах?

A) Стек (Stack)

B) Очередь (Queue)

C) Дерево (Tree)

D) Связный список

5. Что вернет функция `f(4)` ? `def f(n): return 1 if n<=1 else n + f(n-1)`

A) 24

B) 10

C) 12

D) 4

6. Какой метод строки возвращает `-1`, если искомая подстрока не найдена?

A) `index()`

B) `find()`

C) `split()`

D) `count()`

7. Каким является алгоритм сортировки, сравнивающий и меняющий местами соседние элементы?

A) Сортировка выбором

B) Быстрая сортировка

C) Пузырьковая сортировка

D) Сортировка слиянием

8. Если в графе из 5 вершин каждая вершина соединена со всеми остальными, то это:

A) Дерево

B) Полный граф

C) Двудольный граф

D) Несвязный граф

Ответы и разборы вопросов:

1 - B

2 - C

3 - D

4 - B

5 - B

6 - B

7 - C

8 - B

Пояснения:

1) Срез со 2-го символа ('Н'), до 6-го не включительно, шаг 2 → индексы 1, 3, 5 ('Н', '_', '0') → 'H_0'.

5) $f(4) = 4 + f(3) = 4 + 3 + 2 + 1 = 10$.

6) `find()` возвращает `-1`, а `index()` вызывает ошибку `ValueError`.